# Assigned vs unassigned proteins: comparative analysis

**Response to Reviewer 1, comment #3 (~19% unassigned proteins)**

Montes-Ortiz et al.

5/5/26

## Table of contents



## 1 Aim

Characterize the ~19% of *T. lineatum* proteins that were not assigned to any orthogroup by OrthoFinder, to evaluate whether they represent genuine lineage-specific biology or are enriched for potential annotation artefacts (e.g., fragmentary predictions, TE-derived ORFs).

Comparisons between **assigned** (proteins placed in any orthogroup) and **unassigned** (singletons) sets:

- Protein length

- Proportion with any InterProScan domain
- Proportion with Pfam domain
- Number of distinct domains per protein
- Overlap with transposable element (TE) annotations

**Namespace conflicts.** We use explicit `package::function` prefixes because loading `topGO`/`Biostrings`/`GenomicRanges` masks several base/tidyverse functions (`select`, `filter`, `setdiff`, `intersect`, `rename`).

---

## 2 1. Setup

```
# ---- Installation (uncomment only the first time) ----
# install.packages(c("tidyverse", "data.table", "scales", "effsize"))
# BiocManager::install(c("Biostrings", "GenomicRanges", "rtracklayer"))

suppressPackageStartupMessages({
  library(tidyverse)
  library(data.table)
  library(Biostrings)
  library(GenomicRanges)
  library(rtracklayer)
  library(scales)
})

set.seed(42)

out_dir <- file.path("results", "assigned_vs_unassigned")
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
```

---

## 3 2. Parameters

```
# --- EDIT these paths to point to your local files ---
protein_fasta   <- "Trypodendron_lineatum.all.maker.proteins.fasta"
interpro_tsv    <- "Tlin_interpro.tsv"
orthogroups_tsv <- "Orthogroups.tsv"
unassigned_tsv  <- "Orthogroups_UnassignedGenes.tsv"
gene_gff        <- "Trypodendron_lineatum_R3.clean.gff"
te_file         <- "Trypodendron_lineatum.fna.out"

# Column for T. lineatum in the orthogroups files
target_species  <- "Fix_Trypodendron_lineatum"
```

```
# GFF: which feature type carries gene coordinates
feature_type    <- "gene"

# TE overlap: a gene is flagged as TE-derived if >= this % of its locus
# overlaps with an annotated TE
te_overlap_min_pct <- 50
```

---

## 4 3. Build the assigned vs unassigned sets

```
# --- All predicted proteins (universe) ---
prot_seqs <- Biostrings::readAAStringSet(protein_fasta)
names(prot_seqs) <- stringr::word(names(prot_seqs), 1)  # keep only first token

prot_lengths <- tibble::tibble(
  protein_id = names(prot_seqs),
  length_aa  = Biostrings::width(prot_seqs)
)

cat("Total predicted proteins (FASTA):", nrow(prot_lengths), "\n")
```

```
Total predicted proteins (FASTA): 15009
```

```
cat("Example protein IDs:", paste(head(prot_lengths$protein_id, 3), collapse = ", "), "\n")
```

```
Example protein IDs: Tlin14012-RA, Tlin14013-RA, Tlin13837-RA
```

```
# --- Assigned (in Orthogroups.tsv, target species column) ---
og <- readr::read_tsv(orthogroups_tsv, col_types = readr::cols(.default = "c"))
stopifnot(target_species %in% colnames(og))

assigned_ids <- og %>%
  dplyr::pull(!!target_species) %>%
  stringr::str_split(",\\s*") %>%
  unlist() %>%
  stringr::str_trim() %>%
  purrr::discard(~ is.na(.x) || .x == "") %>%
  unique()

cat("Assigned proteins (in orthogroups):", length(assigned_ids), "\n")
```

```
Assigned proteins (in orthogroups): 12115
```

```r
# --- Unassigned (singletons) from OrthoFinder ---
unassigned <- readr::read_tsv(unassigned_tsv, col_types = readr::cols(.default = "c"))

unassigned_ids <- unassigned %>%
  dplyr::pull(!!target_species) %>%
  stringr::str_split(",\\s*") %>%
  unlist() %>%
  stringr::str_trim() %>%
  purrr::discard(~ is.na(.x) || .x == "") %>%
  unique()

cat("Unassigned proteins (singletons):  ", length(unassigned_ids), "\n")
```

```
Unassigned proteins (singletons):   2715
```

```r
# --- Sanity checks ---
overlap_err <- base::intersect(assigned_ids, unassigned_ids)
missing_ids <- base::setdiff(c(assigned_ids, unassigned_ids), prot_lengths$protein_id)

if (length(overlap_err) > 0)
  warning("IDs in both assigned and unassigned: ", length(overlap_err))
if (length(missing_ids) > 0)
  warning("IDs in orthogroups but not in FASTA: ", length(missing_ids),
          " (first 5: ", paste(head(missing_ids, 5), collapse = ", "), ")")

# --- Build master table ---
master <- prot_lengths %>%
  dplyr::mutate(
    group = dplyr::case_when(
      protein_id %in% assigned_ids   ~ "Assigned",
      protein_id %in% unassigned_ids ~ "Unassigned",
      TRUE                           ~ "Other"
    )
  )

master %>% dplyr::count(group) %>% print()
```

```
# A tibble: 3 x 2
  group          n
  <chr>      <int>
1 Assigned   12115
2 Other        179
3 Unassigned  2715
```

```r
pct_unassigned <- 100 * sum(master$group == "Unassigned") / nrow(master)
cat(sprintf("\nUnassigned fraction: %.2f%%\n", pct_unassigned))
```

```
Unassigned fraction: 18.09%
```

---

## 5 4. Parse InterProScan and add annotation features

```
ipr_cols <- c("protein_id", "md5", "length", "analysis",
              "sig_acc", "sig_desc", "start", "end",
              "score", "status", "date",
              "ipr_acc", "ipr_desc", "go_terms", "pathways")

ipr_dt <- data.table::fread(interpro_tsv, sep = "\t", header = FALSE,
                            fill = TRUE, na.strings = c("", "NA", "-"),
                            colClasses = "character", quote = "")
data.table::setnames(ipr_dt, old = names(ipr_dt),
                     new = ipr_cols[seq_len(ncol(ipr_dt))])
ipr <- tibble::as_tibble(ipr_dt); rm(ipr_dt); invisible(gc())

# Per-protein annotation summaries
annot_summary <- ipr %>%
  dplyr::group_by(protein_id) %>%
  dplyr::summarise(
    has_any_ipr    = any(!is.na(ipr_acc)),
    has_pfam       = any(analysis == "Pfam"),
    n_ipr_domains  = dplyr::n_distinct(ipr_acc, na.rm = TRUE),
    n_pfam_domains = dplyr::n_distinct(sig_acc[analysis == "Pfam"], na.rm = TRUE),
    has_go         = any(!is.na(go_terms)),
    .groups = "drop"
  )

master <- master %>%
  dplyr::left_join(annot_summary, by = "protein_id") %>%
  dplyr::mutate(
    has_any_ipr    = tidyr::replace_na(has_any_ipr, FALSE),
    has_pfam       = tidyr::replace_na(has_pfam, FALSE),
    has_go         = tidyr::replace_na(has_go, FALSE),
    n_ipr_domains  = tidyr::replace_na(n_ipr_domains, 0L),
    n_pfam_domains = tidyr::replace_na(n_pfam_domains, 0L)
  )

cat("Annotation coverage:\n")
```

```
Annotation coverage:
```

```
master %>%
  dplyr::filter(group %in% c("Assigned", "Unassigned")) %>%
```

```
  dplyr::group_by(group) %>%
  dplyr::summarise(
    n            = dplyr::n(),
    median_len   = median(length_aa),
    mean_len     = round(mean(length_aa), 1),
    pct_any_ipr  = round(100 * mean(has_any_ipr), 1),
    pct_pfam     = round(100 * mean(has_pfam), 1),
    pct_go       = round(100 * mean(has_go), 1),
    .groups      = "drop"
  ) %>%
  print()
```

```
# A tibble: 2 x 7
  group          n median_len mean_len pct_any_ipr pct_pfam pct_go
  <chr>      <int>      <int>    <dbl>       <dbl>    <dbl>  <dbl>
1 Assigned   12115        385     525.        79.7     77.5   54.7
2 Unassigned  2715         77     101.         4.5      4      2.7
```

---

# 6 5. TE overlap via GFF coordinates

## 6.1 5.1 Load gene GFF and map to protein IDs

Your GFF uses `gene` features with IDs like `Tlin14012`, while your FASTA protein IDs are `Tlin14012-RA` (transcript-level). We build the mapping by also loading `mRNA` features (which have both the mRNA ID matching the protein and the `Parent=gene_id` pointer), then joining.

```
gff_all <- rtracklayer::import(gene_gff)

# --- Genes: their coordinates ---
genes_gr <- gff_all[gff_all$type == feature_type]
S4Vectors::mcols(genes_gr)$gene_id <- as.character(S4Vectors::mcols(genes_gr)$ID)

cat("Genes loaded from GFF:", length(genes_gr), "\n")
```

```
Genes loaded from GFF: 15009
```

```
# --- mRNAs: map each protein_id to its parent gene ---
mrnas <- gff_all[gff_all$type == "mRNA"]
mrna_map <- tibble::tibble(
  mrna_id = as.character(S4Vectors::mcols(mrnas)$ID),
  gene_id = as.character(S4Vectors::mcols(mrnas)$Parent)
) %>%
  dplyr::distinct()
```

```r
cat("mRNA -> gene entries:", nrow(mrna_map), "\n")
```

```
mRNA -> gene entries: 15009
```

```r
cat("Example:\n")
```

```
Example:
```

```r
print(head(mrna_map, 3))
```

```
# A tibble: 3 x 2
  mrna_id      gene_id
  <chr>        <chr>
1 Tlin14012-RA Tlin14012
2 Tlin14013-RA Tlin14013
3 Tlin13837-RA Tlin13837
```

```r
cat("FASTA ID format example:", head(master$protein_id, 1), "\n")
```

```
FASTA ID format example: Tlin14012-RA
```

```r
# Sanity check: does protein_id format == mRNA ID format?
n_match <- sum(master$protein_id %in% mrna_map$mrna_id)
cat(sprintf("\nProtein IDs matching mRNA IDs directly: %d / %d\n",
            n_match, nrow(master)))
```

```
Protein IDs matching mRNA IDs directly: 15009 / 15009
```

```r
# If IDs don't match directly, try stripping common suffixes
if (n_match < 0.5 * nrow(master)) {
  cat("Low match rate. Trying to match after stripping '-RA', '-RB'... suffixes\n")
  master <- master %>%
    dplyr::mutate(gene_id_guess = stringr::str_remove(protein_id, "-R[A-Z]$"))
  n_match2 <- sum(master$gene_id_guess %in% mrna_map$gene_id)
  cat(sprintf("After suffix stripping, matching gene IDs: %d / %d\n",
              n_match2, nrow(master)))
}
```

## 6.2 5.2 TE parsing (RepeatMasker .out)

```
te_ext <- tolower(tools::file_ext(te_file))

if (te_ext %in% c("gff", "gff3", "bed")) {
  te_gr <- rtracklayer::import(te_file)

} else if (te_ext == "out") {
  # RepeatMasker .out: 3-line fixed header, whitespace-delimited,
  # some rows end in a trailing "*". Keep only first 15 columns.
  rm_lines <- readLines(te_file)
  rm_lines <- rm_lines[-c(1, 2, 3)]
  rm_lines <- rm_lines[nchar(trimws(rm_lines)) > 0]

  parts <- strsplit(trimws(rm_lines), "\\s+")
  parts <- lapply(parts, function(x) {
    x <- x[seq_len(min(length(x), 15))]
    length(x) <- 15
    x
  })
  rm_df <- as.data.frame(do.call(rbind, parts), stringsAsFactors = FALSE)
  names(rm_df) <- c("sw_score","pct_div","pct_del","pct_ins",
                    "seqnames","start","end","q_left",
                    "strand","repeat_name","repeat_class",
                    "r_start","r_end","r_left","id")

  rm_df$start <- as.integer(rm_df$start)
  rm_df$end   <- as.integer(rm_df$end)

  te_gr <- GenomicRanges::GRanges(
    seqnames     = rm_df$seqnames,
    ranges       = IRanges::IRanges(start = rm_df$start, end = rm_df$end),
    strand       = ifelse(rm_df$strand == "C", "-", "+"),
    repeat_class = rm_df$repeat_class,
    repeat_name  = rm_df$repeat_name
  )
  cat("Parsed TE records from .out:", length(te_gr), "\n")

} else {
  stop("Unknown TE file format: ", te_ext)
}
```

```
Parsed TE records from .out: 136181
```

```
# Simplify TE class (drop family detail)
S4Vectors::mcols(te_gr)$repeat_class_main <- stringr::str_remove(
  S4Vectors::mcols(te_gr)$repeat_class, "/.*$"
)

cat("\nTE class distribution:\n")
```

TE class distribution:

```
print(sort(table(S4Vectors::mcols(te_gr)$repeat_class_main), decreasing = TRUE))
```

```
 Simple_repeat        Unknown Low_complexity            DNA            LTR
         86980          25568          14044           4664           4336
          LINE             RC      Satellite
           272            211            106
```

```
# Check contig name compatibility
gene_seqs <- unique(as.character(GenomicRanges::seqnames(genes_gr)))
te_seqs   <- unique(as.character(GenomicRanges::seqnames(te_gr)))
common    <- base::intersect(gene_seqs, te_seqs)
cat(sprintf("\nContigs in genes GFF: %d\nContigs in TE .out: %d\nContigs in both: %d\n",
            length(gene_seqs), length(te_seqs), length(common)))
```

```
Contigs in genes GFF: 689
Contigs in TE .out: 833
Contigs in both: 689
```

### 6.3 5.3 Compute gene <-> TE overlap

```
# Optionally exclude simple/low-complexity repeats from the "TE-derived" flag
te_for_overlap <- te_gr[!S4Vectors::mcols(te_gr)$repeat_class_main %in%
                          c("Simple_repeat", "Low_complexity", "Satellite", "rRNA")]
cat("TE records used for overlap (excluding simple/low-complexity):",
    length(te_for_overlap), "\n")
```

```
TE records used for overlap (excluding simple/low-complexity): 35051
```

```
hits <- GenomicRanges::findOverlaps(genes_gr, te_for_overlap)
ov   <- IRanges::pintersect(genes_gr[S4Vectors::queryHits(hits)],
                            te_for_overlap[S4Vectors::subjectHits(hits)])

overlap_bp <- tibble::tibble(
  gene_id = genes_gr$gene_id[S4Vectors::queryHits(hits)],
  overlap = BiocGenerics::width(ov)
) %>%
  dplyr::group_by(gene_id) %>%
  dplyr::summarise(te_overlap_bp = sum(overlap), .groups = "drop")

gene_lengths_bp <- tibble::tibble(
  gene_id        = genes_gr$gene_id,
  gene_length_bp = BiocGenerics::width(genes_gr)
```

```
  ) %>% dplyr::distinct(gene_id, .keep_all = TRUE)

  te_summary <- gene_lengths_bp %>%
    dplyr::left_join(overlap_bp, by = "gene_id") %>%
    dplyr::mutate(
      te_overlap_bp  = tidyr::replace_na(te_overlap_bp, 0),
      te_overlap_pct = 100 * te_overlap_bp / gene_length_bp,
      te_derived     = te_overlap_pct >= te_overlap_min_pct
    )

  # Join master (protein_id) -> mrna_map (gene_id) -> te_summary
  master <- master %>%
    dplyr::left_join(
      mrna_map %>% dplyr::rename(protein_id = mrna_id),
      by = "protein_id"
    ) %>%
    dplyr::left_join(
      te_summary %>% dplyr::select(gene_id, te_overlap_pct, te_derived),
      by = "gene_id"
    )

  cat("\nTE overlap summary:\n")
```

```
TE overlap summary:
```

```
  master %>%
    dplyr::filter(group %in% c("Assigned", "Unassigned")) %>%
    dplyr::group_by(group) %>%
    dplyr::summarise(
      n               = dplyr::n(),
      n_with_gff      = sum(!is.na(te_overlap_pct)),
      pct_te_derived  = round(100 * mean(te_derived, na.rm = TRUE), 2),
      median_overlap  = round(median(te_overlap_pct, na.rm = TRUE), 2),
      .groups         = "drop"
    ) %>%
    print()
```

```
# A tibble: 2 x 5
  group          n n_with_gff pct_te_derived median_overlap
  <chr>      <int>      <int>          <dbl>          <dbl>
1 Assigned   12115      12115           0.5               0
2 Unassigned  2715       2715           0.48              0
```

```
  readr::write_tsv(master, file.path(out_dir, "master_protein_table.tsv"))
```

---

## 7 6. Statistical tests

```
library("effsize")
```

```
a <- master %>% dplyr::filter(group == "Assigned")
u <- master %>% dplyr::filter(group == "Unassigned")

fp <- function(p) {
  if (is.na(p)) "NA"
  else if (p < 1e-300) "< 1e-300"
  else if (p < 1e-4)  sprintf("%.2e", p)
  else                sprintf("%.4f", p)
}

# --- Length: Wilcoxon + Cliff's delta ---
wt_len <- wilcox.test(a$length_aa, u$length_aa, alternative = "two.sided")
cd_len <- effsize::cliff.delta(a$length_aa, u$length_aa)

# --- Proportions: Fisher's exact ---
prop_test <- function(var) {
  tab <- matrix(c(sum(a[[var]]),  sum(!a[[var]]),
                  sum(u[[var]]),  sum(!u[[var]])), nrow = 2, byrow = TRUE)
  ft <- fisher.test(tab)
  tibble::tibble(
    variable     = var,
    assigned_n   = sum(a[[var]]),
    assigned_pct = round(100 * mean(a[[var]]), 2),
    unassig_n    = sum(u[[var]]),
    unassig_pct  = round(100 * mean(u[[var]]), 2),
    odds_ratio   = round(ft$estimate, 3),
    p_value      = ft$p.value
  )
}

prop_results <- dplyr::bind_rows(
  prop_test("has_any_ipr"),
  prop_test("has_pfam"),
  prop_test("has_go")
)

# --- TE-derived: restricted to proteins with GFF mapping ---
a_te <- a %>% dplyr::filter(!is.na(te_derived))
u_te <- u %>% dplyr::filter(!is.na(te_derived))

if (nrow(a_te) > 0 && nrow(u_te) > 0) {
  te_tab <- matrix(c(sum(a_te$te_derived),  sum(!a_te$te_derived),
                     sum(u_te$te_derived),  sum(!u_te$te_derived)),
                   nrow = 2, byrow = TRUE)
```

```
  ft_te  <- fisher.test(te_tab)

  prop_results <- dplyr::bind_rows(
    prop_results,
    tibble::tibble(
      variable     = "te_derived",
      assigned_n   = sum(a_te$te_derived),
      assigned_pct = round(100 * mean(a_te$te_derived), 2),
      unassig_n    = sum(u_te$te_derived),
      unassig_pct  = round(100 * mean(u_te$te_derived), 2),
      odds_ratio   = round(ft_te$estimate, 3),
      p_value      = ft_te$p.value
    )
  )
}

prop_results <- prop_results %>%
  dplyr::mutate(p_adj      = p.adjust(p_value, method = "BH"),
                p_value_fmt= purrr::map_chr(p_value, fp),
                p_adj_fmt  = purrr::map_chr(p_adj, fp))

# --- Length row ---
len_row <- tibble::tibble(
  variable        = "length_aa",
  assigned_n      = nrow(a),
  assigned_median = median(a$length_aa),
  assigned_pct    = NA_real_,
  unassig_n       = nrow(u),
  unassig_median  = median(u$length_aa),
  unassig_pct     = NA_real_,
  odds_ratio      = NA_real_,
  p_value         = wt_len$p.value,
  p_value_fmt     = fp(wt_len$p.value),
  cliffs_delta    = round(as.numeric(cd_len$estimate), 3),
  cd_magnitude    = as.character(cd_len$magnitude)
)

final_stats <- dplyr::bind_rows(
  len_row,
  prop_results %>% dplyr::mutate(
    assigned_median = NA_real_,
    unassig_median  = NA_real_,
    cliffs_delta    = NA_real_,
    cd_magnitude    = NA_character_
  )
) %>%
  dplyr::select(variable,
                assigned_n, assigned_median, assigned_pct,
                unassig_n,  unassig_median,  unassig_pct,
```

```
                odds_ratio, cliffs_delta, cd_magnitude,
                p_value_fmt, p_adj_fmt)

print(final_stats)
```

```
# A tibble: 5 x 12
  variable    assigned_n assigned_median assigned_pct unassig_n unassig_median
  <chr>            <int>           <dbl>        <dbl>     <int>           <dbl>
1 length_aa        12115             385         NA        2715              77
2 has_any_ipr       9657              NA         79.7       121              NA
3 has_pfam          9394              NA         77.5       109              NA
4 has_go            6629              NA         54.7        74              NA
5 te_derived          61              NA          0.5        13              NA
# i 6 more variables: unassig_pct <dbl>, odds_ratio <dbl>, cliffs_delta <dbl>,
#   cd_magnitude <chr>, p_value_fmt <chr>, p_adj_fmt <chr>
```

```
readr::write_tsv(final_stats,
                 file.path(out_dir, "TableSX_assigned_vs_unassigned_stats.tsv"))
```

---

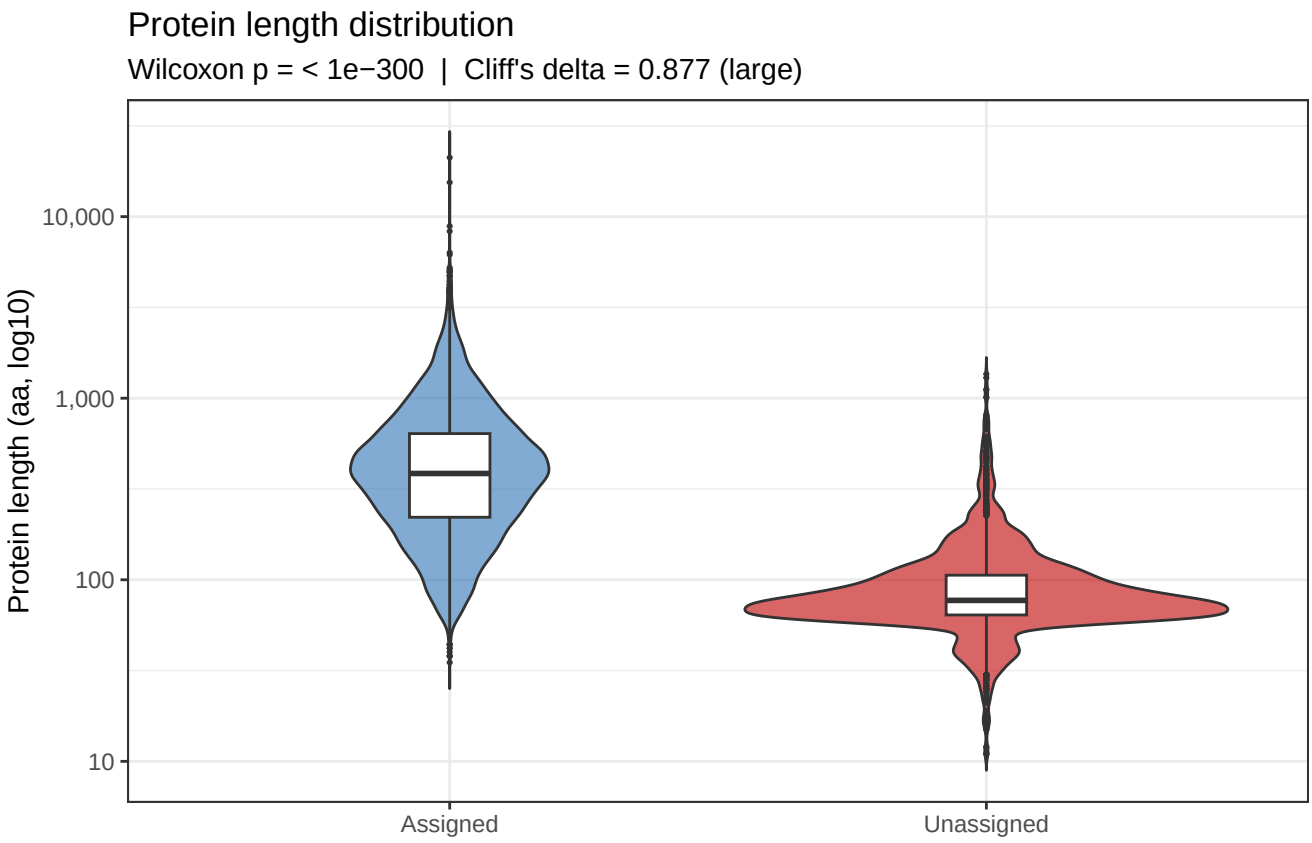
# 8 7. Plots

## 8.1 7.1 Length distribution

```
plot_data <- master %>% dplyr::filter(group %in% c("Assigned", "Unassigned"))

p_len <- ggplot2::ggplot(plot_data,
                         ggplot2::aes(x = group, y = length_aa, fill = group)) +
  ggplot2::geom_violin(alpha = 0.6, trim = FALSE) +
  ggplot2::geom_boxplot(width = 0.15, outlier.size = 0.4, fill = "white") +
  ggplot2::scale_y_log10(labels = scales::comma) +
  ggplot2::scale_fill_manual(values = c(Assigned = "#2E74B5", Unassigned = "#C00000"),
                             guide = "none") +
  ggplot2::labs(x = NULL, y = "Protein length (aa, log10)",
                title = "Protein length distribution",
                subtitle = sprintf("Wilcoxon p = %s  |  Cliff's delta = %.3f (%s)",
                                   fp(wt_len$p.value),
                                   as.numeric(cd_len$estimate),
                                   cd_len$magnitude)) +
  ggplot2::theme_bw(base_size = 11)

print(p_len)
```



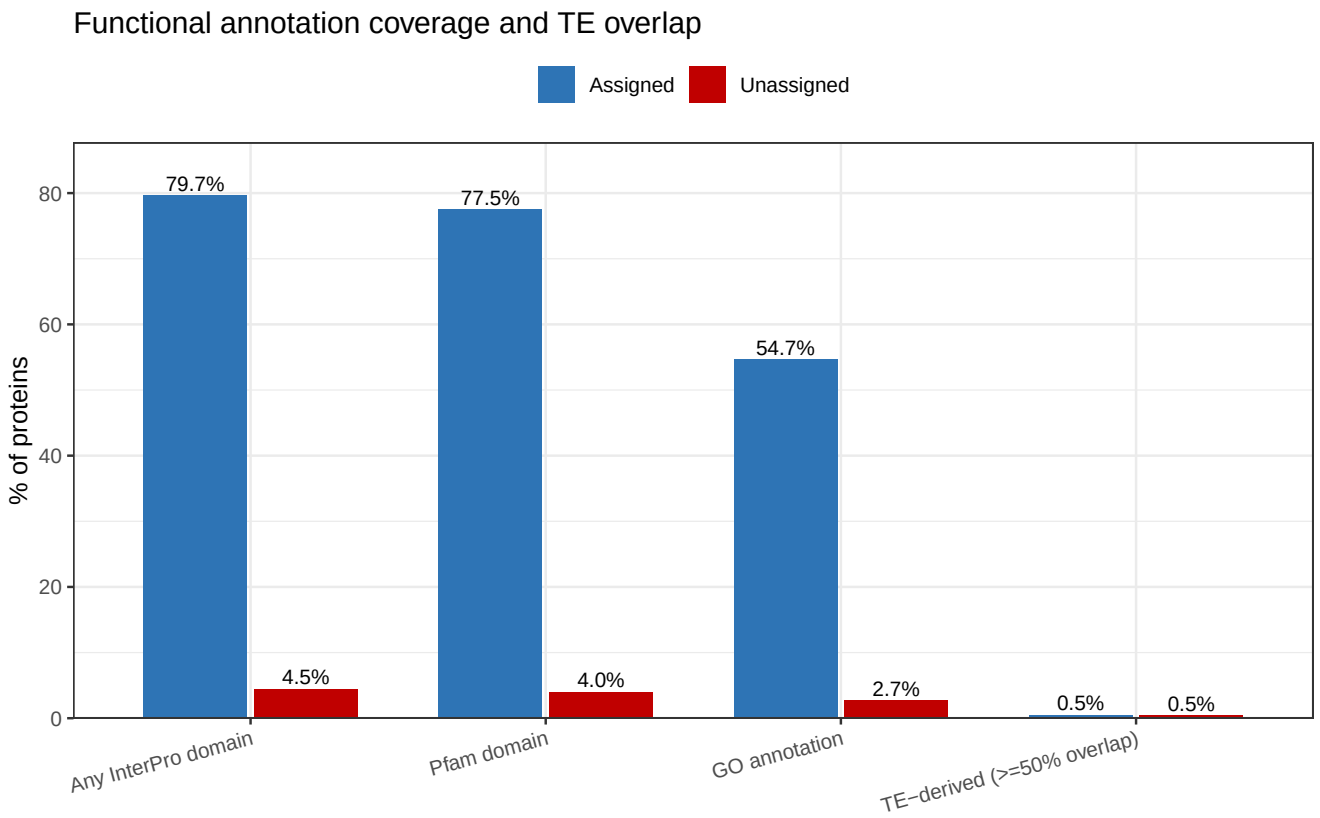

```
ggplot2::ggsave(file.path(out_dir, "FigureSX_length_distribution.png"),
                p_len, width = 7, height = 4.5, dpi = 300)
ggplot2::ggsave(file.path(out_dir, "FigureSX_length_distribution.pdf"),
                p_len, width = 7, height = 4.5)
```

## 8.2 7.2 Annotation and TE overlap proportions

```
plot_prop <- final_stats %>%
  dplyr::filter(variable %in% c("has_any_ipr", "has_pfam", "has_go", "te_derived")) %>%
  dplyr::select(variable, assigned_pct, unassig_pct) %>%
  tidyr::pivot_longer(-variable, names_to = "group", values_to = "pct") %>%
  dplyr::mutate(
    group = dplyr::recode(group, assigned_pct = "Assigned", unassig_pct = "Unassigned"),
    variable = dplyr::recode(variable,
                             has_any_ipr = "Any InterPro domain",
                             has_pfam    = "Pfam domain",
                             has_go      = "GO annotation",
                             te_derived  = sprintf("TE-derived (>=%d%% overlap)", te_overlap_mi
    variable = factor(variable, levels = c("Any InterPro domain", "Pfam domain",
                                           "GO annotation",
                                           sprintf("TE-derived (>=%d%% overlap)", te_overlap_mi
  )
```

```
p_prop <- ggplot2::ggplot(plot_prop,
                          ggplot2::aes(x = variable, y = pct, fill = group)) +
  ggplot2::geom_col(position = ggplot2::position_dodge(0.75), width = 0.7) +
  ggplot2::geom_text(ggplot2::aes(label = sprintf("%.1f%%", pct)),
                     position = ggplot2::position_dodge(0.75),
                     vjust = -0.3, size = 3.2) +
  ggplot2::scale_fill_manual(values = c(Assigned = "#2E74B5", Unassigned = "#C00000"),
                             name = NULL) +
  ggplot2::scale_y_continuous(limits = c(0, NA),
                              expand = ggplot2::expansion(mult = c(0, .1))) +
  ggplot2::labs(x = NULL, y = "% of proteins",
                title = "Functional annotation coverage and TE overlap") +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(legend.position = "top",
                 axis.text.x = ggplot2::element_text(angle = 15, hjust = 1))

print(p_prop)
```




```
ggplot2::ggsave(file.path(out_dir, "FigureSX_annotation_vs_TE.png"),
                p_prop, width = 8, height = 5, dpi = 300)
ggplot2::ggsave(file.path(out_dir, "FigureSX_annotation_vs_TE.pdf"),
                p_prop, width = 8, height = 5)
```

## 9 8. Final summary for the manuscript

```
cat("===== SUMMARY FOR R1 COMMENT #3 =====\n\n")
```

```
===== SUMMARY FOR R1 COMMENT #3 =====
```

```
cat(sprintf("Total predicted proteins:    %d\n", nrow(master)))
```

```
Total predicted proteins:    15009
```

```
cat(sprintf("Assigned (orthogroups):      %d (%.1f%%)\n",
            sum(master$group == "Assigned"),
            100 * sum(master$group == "Assigned") / nrow(master)))
```

```
Assigned (orthogroups):      12115 (80.7%)
```

```
cat(sprintf("Unassigned (singletons):     %d (%.1f%%)\n\n",
            sum(master$group == "Unassigned"),
            100 * sum(master$group == "Unassigned") / nrow(master)))
```

```
Unassigned (singletons):     2715 (18.1%)
```

```
cat("Median protein length:\n")
```

```
Median protein length:
```

```
cat(sprintf("  Assigned:   %d aa\n", median(a$length_aa)))
```

```
  Assigned:   385 aa
```

```
cat(sprintf("  Unassigned: %d aa\n", median(u$length_aa)))
```

```
  Unassigned: 77 aa
```

```
  cat(sprintf("  Wilcoxon p = %s, Cliff's d = %.3f (%s)\n\n",
              fp(wt_len$p.value), as.numeric(cd_len$estimate), cd_len$magnitude))
```

```
  Wilcoxon p = < 1e-300, Cliff's d = 0.877 (large)
```

```
  print(final_stats)
```

```
# A tibble: 5 x 12
  variable    assigned_n assigned_median assigned_pct unassig_n unassig_median
  <chr>            <int>           <dbl>        <dbl>     <int>          <dbl>
1 length_aa        12115             385         NA        2715             77
2 has_any_ipr       9657              NA         79.7       121             NA
3 has_pfam          9394              NA         77.5       109             NA
4 has_go            6629              NA         54.7        74             NA
5 te_derived          61              NA          0.5        13             NA
# i 6 more variables: unassig_pct <dbl>, odds_ratio <dbl>, cliffs_delta <dbl>,
#   cd_magnitude <chr>, p_value_fmt <chr>, p_adj_fmt <chr>
```

---

## 10 9. Session info

```
  sessionInfo()
```

```
R version 4.3.1 (2023-06-16)
Platform: aarch64-apple-darwin20 (64-bit)
Running under: macOS 26.3.1

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRlapack.dylib;  LAPAC

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/Stockholm
tzcode source: internal

attached base packages:
[1] stats4    stats     graphics  grDevices utils     datasets  methods
[8] base

other attached packages:
 [1] effsize_0.8.1        scales_1.4.0         rtracklayer_1.62.0
```

```
 [4] GenomicRanges_1.54.1 Biostrings_2.70.3    GenomeInfoDb_1.38.8
 [7] XVector_0.42.0       IRanges_2.36.0       S4Vectors_0.40.2
[10] BiocGenerics_0.48.1  data.table_1.18.2.1  lubridate_1.9.4
[13] forcats_1.0.1        stringr_1.6.0        dplyr_1.1.4
[16] purrr_1.2.2          readr_2.1.5          tidyr_1.3.1
[19] tibble_3.3.1         ggplot2_4.0.3        tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] SummarizedExperiment_1.32.0 gtable_0.3.6.9000
 [3] rjson_0.2.23                xfun_0.57
 [5] lattice_0.22-9              Biobase_2.62.0
 [7] tzdb_0.5.0                  vctrs_0.7.3
 [9] tools_4.3.1                 bitops_1.0-9
[11] generics_0.1.4              parallel_4.3.1
[13] pkgconfig_2.0.3             Matrix_1.6-5
[15] RColorBrewer_1.1-3          S7_0.2.2
[17] lifecycle_1.0.5             GenomeInfoDbData_1.2.11
[19] compiler_4.3.1              farver_2.1.2
[21] textshaping_1.0.1           Rsamtools_2.18.0
[23] codetools_0.2-20            htmltools_0.5.9
[25] RCurl_1.98-1.18             yaml_2.3.12
[27] pillar_1.11.1               crayon_1.5.3
[29] BiocParallel_1.36.0         DelayedArray_0.28.0
[31] abind_1.4-8                 tidyselect_1.2.1
[33] digest_0.6.37               stringi_1.8.7
[35] restfulr_0.0.16             labeling_0.4.3
[37] fastmap_1.2.0               grid_4.3.1
[39] SparseArray_1.2.4           cli_3.6.5
[41] magrittr_2.0.5              S4Arrays_1.2.1
[43] utf8_1.2.6                  dichromat_2.0-0.1
[45] XML_3.99-0.23               withr_3.0.2
[47] bit64_4.8.0                 timechange_0.3.0
[49] rmarkdown_2.31              matrixStats_1.5.0
[51] bit_4.6.0                   otel_0.2.0
[53] ragg_1.4.0                  hms_1.1.4
[55] evaluate_1.0.5              knitr_1.51
[57] BiocIO_1.12.0               rlang_1.2.0
[59] glue_1.8.1                  vroom_1.6.5
[61] rstudioapi_0.18.0           jsonlite_2.0.0
[63] R6_2.6.1                    systemfonts_1.2.3
[65] MatrixGenerics_1.14.0       GenomicAlignments_1.38.2
[67] zlibbioc_1.48.2
```